

Numerical Methods Lab

Lab 11 – 18 | C Program Outputs

Terminal Output — C

Lab-11: Newton's Forward Difference — First Derivative

Question: Write a Python and C program for calculating first derivative using Newton's forward difference. Data: $x=1, 1.2, 1.4, 1.6$ $y=0, 0.128, 0.544, 1.296$ at $x=1.1$

C Output:

```
big@hell-na:~/Kishor/nm$ gcc -std=c11 "lab11.c" -o lab11 -lm && ./lab11
Enter number of data points: 4
Enter x and y values (x y):
x[0] y[0]: 1 0
x[1] y[1]: 1.2 0.128
x[2] y[2]: 1.4 0.544
x[3] y[3]: 1.6 1.296
Enter the value of x at which derivative is required: 1.1

First derivative at x = 1.1 is 0.6300
big@hell-na:~/Kishor/nm$
```

Lab-12: Newton's Backward Difference — First Derivative

Question: Write a Python and C program for calculating first derivative using Newton's backward difference. Data: $x=1, 1.2, 1.4, 1.6$ $y=0, 0.128, 0.544, 1.296$ at $x=1.1$

C Output:

```
big@hell-na:~/Kishor/nm$ gcc -std=c11 "lab12.c" -o lab12 -lm && ./lab12
Enter number of data points: 4
Enter x and y values (x y):
x[0] y[0]: 1 0
x[1] y[1]: 1.2 0.128
x[2] y[2]: 1.4 0.544
x[3] y[3]: 1.6 1.296
Enter the value of x at which derivative is required: 1.1
First derivative at x = 1.1 is 0.6300
big@hell-na:~/Kishor/nm$
```

Lab-13: Simpson's 1/3 Rule

Question: Write a Python and C program for Composite Simpson's 1/3 Rule. $f(x) = x^2 + 2x + 1$, $x_0=0$, $x_n=4$, segments=4

C Output:

```
big@hell-na:~/Kishor/nm$ gcc -std=c11 "lab13.c" -o lab13 && ./lab13
Function: f(x) = x^2 + 2x + 1
Enter lower limit (x0): 0
Enter upper limit (xn): 4
Enter number of segments: 4
Approximate Integral = 41.333336
big@hell-na:~/Kishor/nm$
```

Lab-14: Simpson's 3/8 Rule

Question: Write a Python and C program for Simpson's 3/8 Rule. $f(x) = x^2 + 2x + 1$, $x_0=0$, $x_n=3$, segments=3

C Output:

```
big@hell-na:~/Kishor/nm$ gcc -std=c11 "lab14.c" -o lab14 && ./lab14
Function: f(x) = x^2 + 2x + 1
Enter lower limit (x0): 0
Enter upper limit (xn): 3
Enter number of segments: 3
Approximate integral using Simpson's 3/8 Rule: 21.000000
big@hell-na:~/Kishor/nm$
```

Lab-15: Romberg Integration

Question: Write a Python and C program for Romberg Integration. Integrand: $\sin(x)/x$, $x_0=0$, $x_n=1$, $p=4$ refinements, $q=3$ extrapolations

C Output:

```
big@hell-na:~/Kishor/nm$ gcc -std=c11 "lab15.c" -o lab15 -lm && ./lab15
Enter lower limit (x0): 0
Enter upper limit (xn): 1
Enter number of trapezoidal refinements (p): 4
Enter number of extrapolations (q): 3
Target Romberg estimate T[4][3]
Approximate integral of sin(x)/x from 0.0 to 1.0: 0.9460830704

Romberg Table (T[i][j]):
T[0]: 0.9207354924 0.0000000000 0.0000000000 0.0000000000
T[1]: 0.9397932848 0.9461458823 0.0000000000 0.0000000000
T[2]: 0.9445135217 0.9460869340 0.9460830041 0.0000000000
T[3]: 0.9456908636 0.9460833109 0.9460830694 0.9460830704
T[4]: 0.9459850299 0.9460830854 0.9460830704 0.9460830704
big@hell-na:~/Kishor/nm$
```

Lab-16: Gaussian Elimination

Question: Write a Python and C program for Gaussian Elimination. System: $x+y+z=6$, $2x+3y+z=14$, $x+2y+3z=14$

C Output:

```
big@hell-na:~/Kishor/nm$ gcc -std=c11 "lab16.c" -o lab16 && ./lab16
Enter order of square matrix (n): 3
Enter coefficients of the matrix:
A[1][1]: 1
A[1][2]: 1
A[1][3]: 1
A[2][1]: 2
A[2][2]: 3
A[2][3]: 1
A[3][1]: 1
A[3][2]: 2
A[3][3]: 3

Enter constants of vector b:
b[1]: 6
b[2]: 14
b[3]: 14

Solution of the system:
x = 0.00
y = 4.00
z = 2.00
big@hell-na:~/Kishor/nm$
```

Lab-17: Gaussian Elimination with Partial Pivoting

Question: Write a Python and C program for Gaussian Elimination with Partial Pivoting. System: $x+y+z=6$, $2x+3y+z=14$, $x+2y+3z=14$

C Output:

```
big@hell-na:~/Kishor/nm$ gcc -std=c11 "lab17.c" -o lab17 -lm && ./lab17
Enter order of square matrix (n): 3
Enter coefficients of 3x3 matrix:
A[1][1] = 1
A[1][2] = 1
A[1][3] = 1
A[2][1] = 2
A[2][2] = 3
A[2][3] = 1
A[3][1] = 1
A[3][2] = 2
A[3][3] = 3

Enter constants vector:
b[1] = 6
b[2] = 14
b[3] = 14

Solution of the system:
x = 0
y = 4
z = 2
big@hell-na:~/Kishor/nm$
```

Lab-18: Gauss-Jordan Elimination

Question: Write a Python and C program for Gauss-Jordan Elimination. System: $x+y+z=6$, $2x+3y+z=14$, $x+2y+3z=14$

C Output:

```
big@hell-na:~/Kishor/nm$ gcc -std=c11 "lab18.c" -o lab18 -lm && ./lab18
Enter order of square matrix (n): 3
Enter coefficients of 3x3 matrix:
A[1][1] = 1
A[1][2] = 1
A[1][3] = 1
A[2][1] = 2
A[2][2] = 3
A[2][3] = 1
A[3][1] = 1
A[3][2] = 2
A[3][3] = 3

Enter constants vector:
b[1] = 6
b[2] = 14
b[3] = 14

Gauss-Jordan Solution:
x = 0
y = 4
z = 2
big@hell-na:~/Kishor/nm$
```